

1. Inleiding

Deze documentatie beschrijft de architectuur en opbouw van de Taxi Livo webapplicatie. Het doel van dit document is om inzicht te geven in de technische structuur, de gebruikte architectuur en de manier waarop de verschillende onderdelen van de applicatie met elkaar samenwerken.

Daarnaast dient deze documentatie als naslagwerk voor ontwikkelaars, beheerders en andere betrokkenen die de applicatie willen begrijpen, onderhouden of verder ontwikkelen. Er wordt ingegaan op de algemene architectuur, de gebruikte technologieën, de projectstructuur en de belangrijkste componenten waaruit de webapplicatie is opgebouwd.

Door de opbouw en ontwerpkeuzes duidelijk te documenteren, wordt het eenvoudiger om nieuwe functionaliteiten toe te voegen, wijzigingen door te voeren en de kwaliteit en onderhoudbaarheid van de Taxi Livo webapplicatie te waarborgen.

2. Projectstructuur

De Taxi Livo webapplicatie is ontwikkeld met React als frontend-framework. De applicatie maakt gebruik van JavaScript (JSX) voor de logica, HTML (via JSX) voor de structuur en CSS voor de vormgeving. Voor de backend wordt gebruikgemaakt van Supabase, dat verantwoordelijk is voor de database, authenticatie en API-functionaliteiten. De applicatie wordt gebouwd en gebundeld met Vite, wat zorgt voor snelle ontwikkeling en een efficiënte build.

De projectstructuur is als volgt opgebouwd:

```
taxi-app-1.0/
|
|— .vscode/
|— dist/
|— docs/
|— node_modules/
|— public/
|— src/
|   |— assets/
|   |— components/
|   |— utils/
|   |— App.jsx
|   |— index.css
```

```
|
| |
| | ├── main.jsx
| | └── supabase.js
|
| ├── supabase/
| ├── .env
| ├── .gitignore
| ├── deno.lock
| ├── eslint.config.js
| ├── generate-icons.js
| ├── index.html
| ├── package.json
| ├── package-lock.json
| ├── README.md
| ├── verify-supabase.js
| └── vite.config.js
```

Toelichting per map

`.vscode`

Bevat instellingen die specifiek zijn voor Visual Studio Code, zoals aanbevolen extensies en projectinstellingen. Deze map heeft geen invloed op de werking van de webapplicatie.

dist

Deze map bevat de gecompileerde productieversie van de applicatie. Wanneer het project wordt gebouwd met Vite (npm run build), worden alle bestanden hierin geplaatst. Deze map wordt gebruikt voor deployment naar een webserver.

docs

In deze map bevindt zich alle technische documentatie van het project, waaronder de architectuur, installatiehandleidingen en uitleg over de applicatie.

node_modules

Bevat alle externe packages en afhankelijkheden die via npm zijn geïnstalleerd. Deze map wordt automatisch gegenereerd en hoeft niet handmatig aangepast te worden.

public

Hierin staan statische bestanden die rechtstreeks beschikbaar zijn voor de browser, zoals afbeeldingen, favicon-bestanden en andere resources die niet door React worden verwerkt.

De src map

De map src bevat de volledige broncode van de React-applicatie.

assets

Bevat afbeeldingen, iconen, logo's en overige statische bestanden die binnen de applicatie worden gebruikt.

components

Deze map bevat alle React-componenten waaruit de gebruikersinterface is opgebouwd. Door de applicatie op te delen in componenten blijft de code overzichtelijk en herbruikbaar.

ActionDialog

Verantwoordelijk voor het tonen van bevestigingsvensters of acties die door de gebruiker moeten worden bevestigd.

Bestanden:

ActionDialog.jsx
ActionDialog.css
Login

Bevat het inlogschermb van de applicatie. Hier loggen chauffeurs of gebruikers in via Supabase Authenticatie.

Bestanden:

Login.jsx
Login.css
MyTripLogs

Toont een overzicht van de ritten van een chauffeur. Vanuit deze component kunnen ritgegevens worden bekeken.

Bestanden:

MyTripLogs.jsx
MyTripLogs.css
Navigation

Bevat de navigatie van de applicatie, zoals het menu of de navigatiebalk tussen de verschillende schermen.

Bestanden:

Navigation.jsx

Navigation.css
RittenstaatForm

Dit is één van de belangrijkste componenten binnen de applicatie. Hier worden de ritgegevens ingevuld en verwerkt voordat deze worden opgeslagen in Supabase.

Bestanden:

RittenstaatForm.jsx
RittenstaatForm.css
SignatureDialog

Verzorgt het plaatsen van een digitale handtekening wanneer een rit wordt afgerond of bevestigd.

Bestanden:

SignatureDialog.jsx
SignatureDialog.css
utils

Bevat hulpfuncties die op meerdere plaatsen binnen de applicatie worden gebruikt. Hierdoor wordt dubbele code voorkomen en blijft de applicatie onderhoudbaar.

App.jsx

Dit is de hoofdcomponent van de React-applicatie. Hier worden de verschillende componenten samengevoegd en wordt bepaald welke onderdelen zichtbaar zijn.

main.jsx

Het startpunt van de React-applicatie. Hier wordt React gekoppeld aan het HTML-document en wordt App.jsx geladen.

index.css

Bevat de algemene styling die voor de gehele applicatie geldt.

supabase.js

Bevat de configuratie voor de verbinding met Supabase. Hier wordt de client geïnitieerd zodat de applicatie verbinding kan maken met de database en authenticatie.

Overige bestanden
supabase/

Deze map bevat Supabase-gerelateerde configuratiebestanden en eventueel Edge Functions of migraties die bij het project horen.

.env

Bevat omgevingsvariabelen, zoals:

Supabase URL

Supabase API Key

Gevoelige informatie wordt hierdoor buiten de broncode gehouden.

.gitignore

Geeft aan welke bestanden niet worden opgeslagen in Git, zoals:

node_modules

.env

dist

eslint.config.js

Configuratiebestand voor ESLint. Dit hulpmiddel controleert de JavaScript-code op fouten en zorgt voor een consistente programmeerstijl.

generate-icons.js

Script dat wordt gebruikt om iconen automatisch te genereren of te verwerken voor de applicatie.

index.html

De HTML-pagina waarin de React-applicatie wordt geladen. React neemt vervolgens de inhoud van deze pagina over.

package.json

Bevat alle projectinformatie, afhankelijkheden en beschikbare scripts, zoals:

npm install

npm run dev

npm run build

package-lock.json

Legt exact vast welke versies van alle npm-packages zijn geïnstalleerd zodat iedere ontwikkelaar dezelfde afhankelijkheden gebruikt.

README.md

Algemene documentatie van het project, zoals installatie-instructies en informatie voor ontwikkelaars.

verify-supabase.js

Script waarmee gecontroleerd kan worden of de verbinding met Supabase correct functioneert.

vite.config.js

Configuratiebestand van Vite. Hier worden instellingen beheerd voor de ontwikkelserver, buildproces en eventuele plugins.

2. Authenticatie

Dit hoofdstuk beschrijft beknopt hoe de inlog- en wachtwoordherstel functionaliteit is geïmplementeerd met behulp van Supabase. Dit is bedoeld als referentie voor toekomstige developers.

1. Supabase Configuratie (`src/supabase.js`)

De Supabase client wordt geïnitieerd via `createClient` vanuit `@supabase/supabase-js`. De benodigde credentials (`VITE\_SUPABASE\_URL` en `VITE\_SUPABASE\_PUBLISHABLE\_KEY`) worden uit de `.env` environment variables gehaald.

2. Inloggen (`src/App.jsx` & `src/components/Login.jsx`)

- **UI Component:** `Login.jsx` bevat alle formulieren (inloggen, wachtwoord vergeten, wachtwoord instellen) en beheert zijn eigen state voor de huidige weergave.
- **Logica:** De daadwerkelijke inlogfunctie wordt in `App.jsx` gedefinieerd (`handleLogin`) en gebruikt de Supabase SDK: `supabase.auth.signInWithPassword({ email, password })`.
- **Sessiebeheer:** `App.jsx` luistert naar authenticatie-events via `supabase.auth.onAuthStateChange`. Bij een succesvolle login wordt de sessie herkend, de state `isLoggedIn` wordt op `true` gezet, en dit wordt bewaard in `localStorage` (`taxilivo\_auth`) om sessies te behouden bij het herladen van de pagina.

3. Wachtwoord Vergeten (Herstellink aanvragen) (`src/components/Login.jsx`)

- In de "Wachtwoord vergeten" view vult de gebruiker een e-mailadres in.
- De functie `handleForgotPasswordRequest` in `Login.jsx` roept `supabase.auth.resetPasswordForEmail(email)` aan.
- Supabase handelt het versturen van de beveiligde e-mail met de herstellink af op basis van de projectinstellingen in het Supabase dashboard.

4. Wachtwoord Wijzigen / Herstellen (`src/components/Login.jsx` & `src/App.jsx`)

- **Detectie van de Herstellink:** Zodra de gebruiker op de link uit de e-mail klikt en terugkeert naar de app, detecteert `App.jsx` dit via de URL (recovery token) of via het `PASSWORD\_RECOVERY` event vanuit Supabase.
- De state `isRecovering` wordt op `true` gezet, waardoor `Login.jsx` direct opent in de view `update\_password`.
- **Nieuw Wachtwoord Opslaan:** De functie `handlePasswordUpdateSubmit` in `Login.jsx` voert de update uit via `supabase.auth.updateUser({ password: newPassword })`.
- **Afronding:** Zodra de update slaagt, wordt de callback `handlePasswordUpdatedComplete` vanuit `App.jsx` aangeroepen. Deze logt de gebruiker direct weer uit (`supabase.auth.signOut()`) ter beveiliging, en verzoekt de gebruiker opnieuw in te loggen met het nieuwe wachtwoord.

Foutafhandeling

Specifieke Supabase error messages (bijv. rate limits, ongeldige inloggegevens, of netwerkfouten) worden in `Login.jsx` via `try-catch` blokken onderschept en vertaald naar duidelijke, Nederlandstalige foutmeldingen voor de gebruiker.

3. Rittenstaten en Pakbonnen - Architectuur

Dit document beschrijft de architectuur en datastroom van de Rittenstaten (inclusief Pakbonnen) vanaf de invoer op de homepage tot de weergave en opslag in de "Mijn Rittenstaten" pagina.

1. De Homepage (`src/components/RittenstaatForm.jsx`)

De homepage is in feite het `RittenstaatForm` component. Hier vult de chauffeur zijn ritten en pakbonnen in.

- **State Management:** Het formulier beheert lokaal verschillende states:
 - `headerLeft` & `headerRight`: De algemene informatie (zoals kenteken, datum, naam, kilometerstanden).
 - `trips`: Een array van standaard 17 ritten.
 - `pakbonnen`: Een dynamische array van pakbonnen. Chauffeurs kunnen pakbonnen toevoegen via `addPakbon` en verwijderen.
 - `signature`: De handtekening (opgeslagen als DataURL via een apart `SignatureDialog` component).
- **Auto-Save (Concept):** Elke keer als een veld (of pakbon) verandert, vuurt de `useEffect` hook de functie `onFormChange` af. Deze functie (geleverd door `App.jsx`) slaat de data direct op in de `localStorage` (`taxilivo_draft`). Hierdoor raakt een chauffeur geen data kwijt als de app wordt afgesloten.

2. Opslaan & Synchronisatie (`src/App.jsx`)

De acties "Opslaan" en "Verzenden" vanuit het `RittenstaatForm` worden afgehandeld in `App.jsx`. Dit vormt de brug tussen de UI en de database (Supabase).

- **Lokaal Opgeslagen:** Bij het opslaan (`handleSaveLog`) wordt de rittenstaat als eerste direct toegevoegd aan de lokale React state (`logs`), zodat de gebruiker direct feedback krijgt ("Rittenstaat lokaal opgeslagen").
- **Cloud Synchronisatie:** Vervolgens start de achtergrond-synchronisatie met Supabase:
 - Het parent object wordt omgezet (via `mapStateToDbForm`) en weggeschreven naar de tabel `forms` met een `.upsert()`.
 - De eventuele gekoppelde pakbonnen worden gekoppeld aan de `form_id` en weggeschreven naar de tabel `pakbonnen` via `.upsert()`.
 - Oude/verwijderde pakbonnen (die lokaal niet meer bestaan) worden netjes opgeruimd uit Supabase (`.delete()`).
- **Verzenden (Inzenden):** Bij het klikken op "Verzenden" of via de functie `handleSendLog` wordt er éérst een PDF gegenereerd en verzonden via een Supabase Edge Function (`send-email`). Bij succes krijgt de log de status `submitted`, wordt deze in Supabase geüpdatet, en wordt de lokale "draft" uit de opslag gewist.

3. Mijn Rittenstaten (`src/components/MyTripLogs.jsx`)

Dit component is verantwoordelijk voor het weergeven van de historie. De data (`logs`) wordt door `App.jsx` uit Supabase gehaald zodra de gebruiker inlogt, waarbij een relationele query (`forms` inclusief `pakbonnen`) wordt uitgevoerd.

- **Weergave & Sortering:** De logs worden chronologisch gesorteerd (`sortedLogs`) en getoond in logkaarten.
- **Informatie & Status:** Elke kaart toont direct de datum, chauffeur, wagen, en handige badges:
 - De status (Opgeslagen vs. Ingezonden).
 - Een pakbon indicator (b.v. "Pakbonnen (2)").
- **Interacties per logkaart:**
 - **Open/Bewerken:** Laadt de log opnieuw in in het `RittenstaatForm` als 'draft', mits deze nog niet is ingezonden (status `saved`).
 - **Inzenden:** Kan alsnog vanuit dit overzicht via de knop 'Inzenden' (`onSubmit`), wat weer de e-mail verstuurt en de status naar `submitted` aanpast.
 - **Verwijderen:** Verwijdert de rittenstaat (`handleDeleteLog`). Allereerst visueel lokaal, vervolgens op de achtergrond definitief uit Supabase.
- **Bulk Acties:** Gebruikers kunnen via checkboxes meerdere rittenstaten selecteren om deze bijvoorbeeld in één keer te downloaden als een samengevoegde PDF, of bulk te verwijderen (`handleDeleteLogs`).

4. Het Verzenden van een Rittenstaat (Supabase Edge Functions)

Dit onderdeel beschrijft de specifieke stroom wanneer een chauffeur ervoor kiest om een rittenstaat (al dan niet met bijbehorende pakbonnen) definitief in te zenden. Deze architectuur is zo opgezet om server-side acties (zoals e-mailen) veilig via **Supabase Edge Functions** uit te voeren.

1. Wat gebeurt er als je op "Verzenden" klikt?

De verzend-logica bevindt zich primair in de functie `submitTripLog()` en de bijbehorende handlers in `src/App.jsx`.

1. **Lokale PDF Generatie:**

Voordat er contact wordt gezocht met de server, gebruikt de applicatie `generatePdfBlob(logData)` (uit `src/Utils/pdfGenerator.js`) om de rittenstaat en de pakbonnen in de browser om te zetten naar een binaire PDF Blob.

2. **Payload Opbouwen:**

Om overhead (zoals base64 encoding) te voorkomen, wordt de gegenereerde PDF samen met wat metadata (wagennummer, naam, datum) in een native `FormData` object gestopt.

3. **Supabase Edge Function Aanroepen:**

De app vuurt `supabase.functions.invoke('send-email', { body: formData })` af. Dit triggert de beveiligde Edge Function op de Supabase server.

4. **Server-side Afhandeling met Resend:**

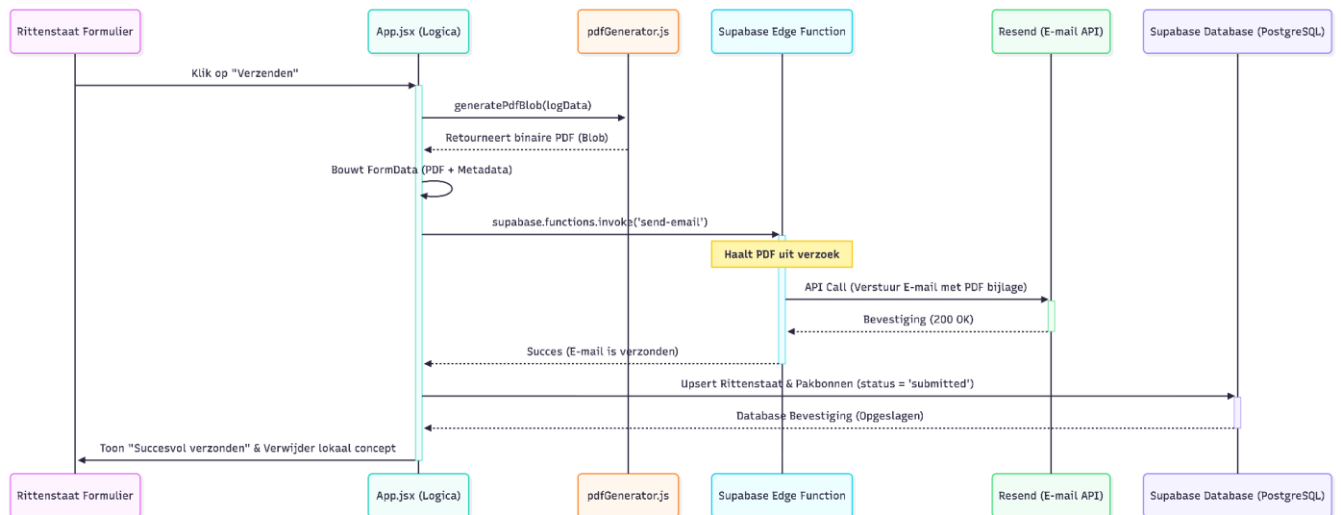
De Edge Function haalt de PDF uit het `FormData` object en verstuurt deze via de e-mail service **Resend**, die de betrouwbare aflevering van de e-mail inclusief PDF-bijlage bij de centrale verzorgt.

5. **Database Synchronisatie:**

Alleen wanneer de e-mail succesvol verzonden is, wordt de status in de React app omgezet naar `submitted`. Vervolgens vindt er een `.upsert()` naar de Supabase Database plaats om de log definitief op te slaan met de status `submitted_at = now()`. Als allerlaatste wordt de lokale cache (`taxilivo_draft` in `localStorage`) netjes opgeschoond.

2. Visuele Weergave (Dataflow)

Hieronder zie je op een simpele manier (middels een Mermaid sequence diagram) de reis die de data aflegt tijdens het inzenden:



``mermaid
sequenceDiagram

participant UI as Rittenstaat Formulier
participant App as App.jsx (Logica)
participant PDF as pdfGenerator.js
participant Edge as Supabase Edge Function
participant Resend as Resend (E-mail API)
participant DB as Supabase Database (PostgreSQL)

UI->>App: Klik op "Verzenden"

activate App

App->>PDF: generatePdfBlob(logData)

PDF-->>App: Retourneert binaire PDF (Blob)

App->>App: Bouwt FormData (PDF + Metadata)

App->>Edge: supabase.functions.invoke('send-email')

activate Edge

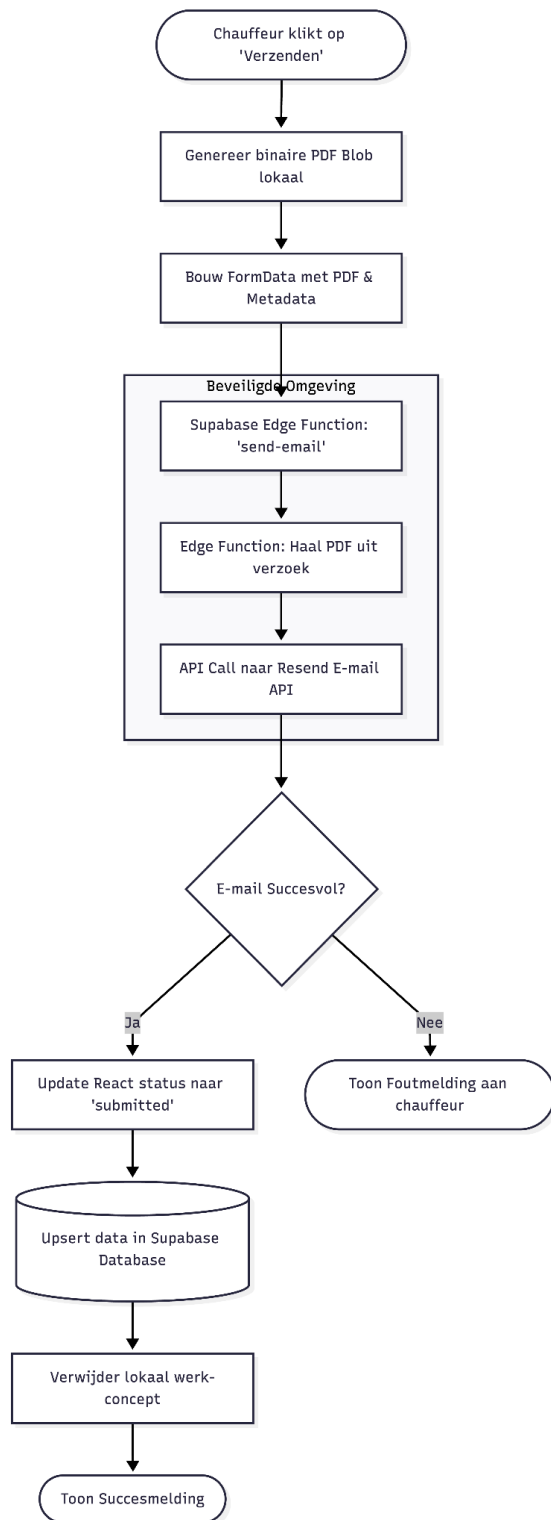
Note over Edge: Haalt PDF uit verzoek

Edge->>Resend: API Call (Verstuur E-mail met PDF bijlage)

activate Resend

3. Stroomschema (Flowchart)

Naast het interactie-diagram hierboven, is hier het logische stappenplan (inclusief foutafhandeling) in de vorm van een flowchart:



```

```mermaid
flowchart TD
 Start([Chauffeur klikt op 'Verzenden']) --> GenPDF[Genereer binaire PDF Blob lokaal]
 GenPDF --> FormData[Bouw FormData met PDF & Metadata]
 FormData --> EdgeCall[Supabase Edge Function: 'send-email']

 subgraph Server-Side [Beveiligde Omgeving]
 EdgeCall --> Extract[Edge Function: Haal PDF uit verzoek]
 Extract --> Resend[API Call naar Resend E-mail API]
 end

 Resend --> SuccessCheck{E-mail Succesvol?}

 SuccessCheck -- Ja --> UpdateStatus[Update React status naar 'submitted']
 UpdateStatus --> UpsertDB[(Upsert data in Supabase Database)]
 UpsertDB --> ClearDraft[Verwijder lokaal werk-concept]
 ClearDraft --> EndSuccess([Toon Succesmelding])

 SuccessCheck -- Nee --> EndError([Toon Foutmelding aan chauffeur])
```

```

5. Database Architectuur & Connectie Overzicht

Dit document geeft een totaaloverzicht van de interactie tussen de React frontend (Web App), het Supabase platform (API & Auth), en de onderliggende PostgreSQL database. Het doel is om een volgende developer direct inzicht te geven in de volledige datastroom: van inloggen tot aan het opslaan en definitief verzenden van een rittenstaat.

De 3 Lagen van de Architectuur

1. **Frontend (React Web App):** De applicatie in de browser van de gebruiker. Hier vinden de interacties plaats, wordt de status lokaal bijgehouden (in React state en `localStorage`), en worden de binaire bestanden (PDF's) gegenereerd.
2. **Supabase (Backend-as-a-Service):** Dit is de tussenlaag. Supabase levert direct bruikbare API's voor authenticatie (`Supabase Auth`), een REST API voor de database (`PostgREST`), en de mogelijkheid om server-side code te draaien (`Edge Functions`).
3. **PostgreSQL (Database):** De robuuste, relationele database onder de motorkap van Supabase. Hierin leven de tabellen zoals `forms` (rittenstaten), `pakbonnen`, en de door Supabase afgeschermdde inloggegevens (`auth.users`).

De Datastroom (User Journey)

1. **Inloggen:**

Wanneer de gebruiker de app opent en inlogt, stuurt de app het e-mailadres en wachtwoord naar **Supabase Auth**. Supabase controleert dit veilig in de **PostgreSQL** `auth.users` tabel. Bij succes krijgt de frontend een sessie (JWT token) terug, waarmee de gebruiker geautoriseerd is voor volgende acties.

2. **Ophalen Historie (Mijn Rittenstaten):**

Zodra de gebruiker is ingelogd, doet de applicatie een verzoek naar de database om eerdere logs op te halen. Dankzij de krachtige relationele opzet van Supabase worden met één query de `forms` én de gekoppelde `pakbonnen` uit PostgreSQL gehaald.

3. **Invullen & Opslaan (Rittenstaat):**

Wanneer de gebruiker een rittenstaat en/of pakbon invult en op "Opslaan" drukt (of wanneer auto-save triggert (localStorage)), wordt er via de **Supabase API** een zogenaamde `.upsert()` uitgevoerd (Update of Insert). Deze data stroomt direct door naar de `public.forms` en `public.pakbonnen` tabellen in **PostgreSQL**.

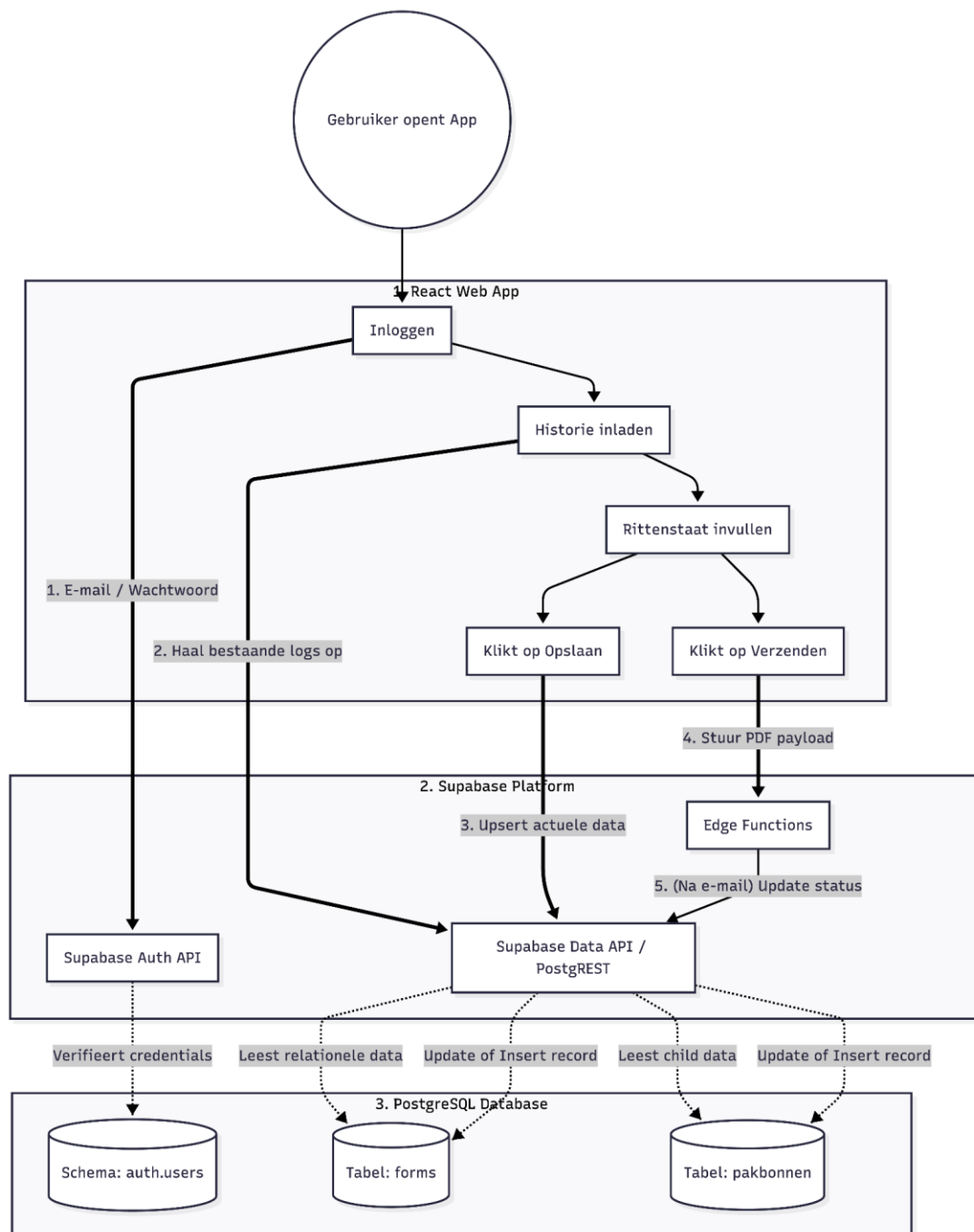
4. **Verzenden:**

Wanneer de rittenstaat compleet is en op "Verzenden" wordt gedrukt, gaat het via een omweg om e-mailafhandeling mogelijk te maken. De app bouwt lokaal een PDF, en stuurt deze naar de

Supabase Edge Function. Nadat de e-mail succesvol is verzonden via Resend, zal de applicatie een laatste update naar de **PostgreSQL** database sturen via de API om de status op `'submitted'` te zetten.

Visuele Weergave (Flowchart)

In onderstaand schema zie je de levenscyclus van de data, waarbij de scheiding tussen het Supabase platform (de API's) en de daadwerkelijke PostgreSQL Database duidelijk in beeld is gebracht.



```mermaid



RestAPI -. "Update of Insert record" .-> DB\_Pakbonnen

%% Connecties Verzenden

Verzenden == "4. Stuur PDF payload" ==> Edge

Edge -- "5. (Na e-mail) Update status" --> RestAPI

...

## 6. Database schema

In dit hoofdstuk wordt het databaseschema van de Taxi Livo webapplicatie beschreven. De applicatie maakt gebruik van een **PostgreSQL-database** die wordt beheerd door **Supabase**. De database is ontworpen om rittenstaten, pakbonnen en gebruikersgegevens op een gestructureerde en betrouwbare manier op te slaan.

Het schema bestaat uit drie hoofdtabellen: `auth.users`, `forms` en `pakbonnen`. De tabel `auth.users` bevat de gegevens van geauthenticeerde gebruikers, terwijl `forms` de rittenstaten opslaat en `pakbonnen` de bijbehorende pakbongegevens bevat. Door middel van primaire en vreemde sleutels zijn de tabellen met elkaar verbonden, waardoor de relaties tussen gebruikers, rittenstaten en pakbonnen worden gewaarborgd.

| auth.users |       |    |             |
|------------|-------|----|-------------|
| uuid       | id    | PK | Primary Key |
| string     | email |    | User Email  |

| forms     |                |    |                           |
|-----------|----------------|----|---------------------------|
| uuid      | id             | PK | Primary Key               |
| uuid      | user_id        | FK | References auth.users(id) |
| timestamp | created_at     |    | Created Timestamp         |
| timestamp | updated_at     |    | Updated Timestamp         |
| timestamp | submitted_at   |    | Submission Timestamp      |
| text      | status         |    | saved   submitted         |
| integer   | km_stand_begin |    | Start Odometer            |
| integer   | km_stand_eind  |    | End Odometer              |
| numeric   | liters_getankt |    | Fuel Liters               |
| text      | ruimte_kantoor |    | Office Notes              |
| text      | wagen_nummer   |    | Vehicle Number            |
| text      | kenteken       |    | License Plate             |
| text      | naam           |    | Driver Name               |
| date      | datum          |    | Shift Date                |
| text      | dag            |    | Day of Week               |
| time      | werktijd_van   |    | Shift Start Time          |
| time      | werktijd_tot   |    | Shift End Time            |
| text      | signature      |    | Driver Signature Data     |
| jsonb     | rides          |    | JSONB Array of Rides      |

| pakbonnen |             |    |                                             |
|-----------|-------------|----|---------------------------------------------|
| uuid      | id          | PK | Primary Key                                 |
| uuid      | form_id     | FK | References forms(id) ON DELETE CASCADE      |
| uuid      | user_id     | FK | References auth.users(id) ON DELETE CASCADE |
| text      | chauffeur   |    | Chauffeur Name                              |
| text      | kenteken    |    | Vehicle License Plate                       |
| text      | scheepsnaam |    | Ship/Vessel Name                            |
| text      | bedrijf     |    | Client Company Name                         |
| text      | naam        |    | Customer Recipient Name                     |
| text      | signature   |    | Customer Signature Data                     |
| text      | van         |    | Pickup Location                             |
| text      | naar        |    | Destination Location                        |
| date      | datum       |    | Delivery Date                               |
| time      | tijd        |    | Delivery Time                               |
| numeric   | ritprijs    |    | Trip Price                                  |
| timestamp | created_at  |    | Created Timestamp                           |
| timestamp | updated_at  |    | Updated Timestamp                           |

erDiagram

```
"auth.users" {
 uuid id PK "Primary Key"
 string email "User Email"
}
forms {
 uuid id PK "Primary Key"
 uuid user_id FK "References auth.users(id)"
 timestamp created_at "Created Timestamp"
 timestamp updated_at "Updated Timestamp"
 timestamp submitted_at "Submission Timestamp"
 text status "saved | submitted"
 integer km_stand_begin "Start Odometer"
 integer km_stand_eind "End Odometer"
 numeric liters_getankt "Fuel Liters"
 text ruimte_kantoor "Office Notes"
 text wagen_nummer "Vehicle Number"
 text kenteken "License Plate"
 text naam "Driver Name"
 date datum "Shift Date"
 text dag "Day of Week"
 time werktijd_van "Shift Start Time"
 time werktijd_tot "Shift End Time"
 text signature "Driver Signature Data"
 jsonb rides "JSONB Array of Rides"
}
pakbonnen {
 uuid id PK "Primary Key"
 uuid form_id FK "References forms(id) ON DELETE CASCADE"
 uuid user_id FK "References auth.users(id) ON DELETE CASCADE"
 text chauffeur "Chauffeur Name"
 text kenteken "Vehicle License Plate"
 text scheepsnaam "Ship/Vessel Name"
 text bedrijf "Client Company Name"
 text naam "Customer Recipient Name"
 text signature "Customer Signature Data"
 text van "Pickup Location"
 text naar "Destination Location"
 date datum "Delivery Date"
 time tijd "Delivery Time"
 numeric ritprijs "Trip Price"
 timestamp created_at "Created Timestamp"
 timestamp updated_at "Updated Timestamp"
}
```